Multi-agent Underwriting Assignment Packet
Design, tracing, PII handling, traces, cost, failure, CI

Standalone unit tests live in tests/unit/test_*_agent.py.
Paste terminal screenshots of pytest and GitHub Actions
around the captured text below if the rubric requires photos.

==========
1. State and memory design decision and rationale
Source: docs/state_memory_design.md
==========
State and Memory Design

Decision

The underwriting pipeline uses a case-scoped shared state model with
controlled agent access and deterministic orchestration.

Each underwriting application is assigned a unique case identifier.
Information produced during processing is persisted as part of that case
rather than being stored in global application memory or relying on an
LLM to remember previous interactions.

Rationale

Insurance underwriting is a multi-stage process that may span multiple
agent executions and potentially multiple sessions. Intake information,
enrichment results, risk assessments, and recommendations therefore need
to remain available throughout the lifecycle of a case.

A shared case state provides a consistent source of truth while explicit
agent contracts limit which portions of the state each component may
produce or consume.

This approach was selected over pure message passing because case data may
need to be retrieved after the original processing step has completed.
However, agents do not receive unrestricted access to mutable global state.
The orchestrator controls execution and provides each agent only the data
required for its responsibility.

State Ownership

Component	Reads	Produces
Intake Agent	Raw application	Validated application
Enrichment Agent	Validated application	Enrichment evidence
Risk Scoring Agent	Application, enrichment evidence	Risk assessment
Recommendation Agent	Application, enrichment evidence, risk assessment	Recommendation

Short-Term Memory

Short-term memory exists only during an individual agent invocation.

Examples include:

- the prompt sent to the model
- relevant case information supplied to that invocation
- temporary tool results
- intermediate reasoning context

Short-term memory is never shared globally between cases and is discarded
after the invocation completes.

Persistent Case State

Information required across workflow stages is stored in the case record.

The case state contains:

- case metadata
- validated application
- enrichment evidence
- risk assessment
- recommendation
- workflow status
- failure information

For the lab implementation, SQLite will provide durable local persistence
behind a repository abstraction. A production implementation could replace
this adapter with an enterprise transactional database without changing
agent business contracts.

Case Isolation

Every application receives a unique case ID.

Agents operate only on state associated with the current case. No global
conversation history or shared applicant context is used.

This prevents information from one applicant from leaking into another
applicant’s processing context.

Case isolation will also be verified through automated tests.

Identifiers and Repeat Submissions

Identifier	Assigned by	Purpose
application_id	Server, at intake	Identifies one request for coverage
case_id / trace_id	Server, at case creation	Identifies one underwriting run and its traces
ssn	Applicant	Strong identifier used to search external evidence
applicant_key	Server (hash of ssn)	Finds other cases for the same person

| idempotency_key | Client (optional) | Makes a resubmitted request safe |

Intake rejects a client-supplied application_id.

External evidence providers are searched by SSN, not by any insurer identifier. Name and date of birth alone are not unique, so medical, prescription, and financial records found by SSN are only attributed to the applicant when the record’s date of birth also matches; otherwise the evidence is marked as a conflict for human review. The identity provider reports name, date of birth, and state matches itself.

Tracing and PII

Every stage attempt writes one trace record. Together they answer, for one case:

Question	Trace fields
Which agent ran?	stage, component
What did it read, and from whom?	received, input_sources, tools_used
What did it produce and write to shared state?	produced, state_writes
When, and for how long?	started_at, finished_at, duration_ms, llm_latency_ms
What did it cost?	model, input_tokens, output_tokens, estimated_cost_usd
Did it fail or retry?	status, attempt, error_type, error_message

input_sources names the producer of each received value: user_input for the submission, case_repository for the applicant’s existing approved coverage, otherwise the agent that wrote it. LLM usage is metered per attempt, so an attempt that fails after a model call (for example on unparseable output) is still costed.

The case repository is the only store that holds applicant PII, and it needs encryption at rest and access control in production. Everything else is minimized:

- LLM prompts never include SSN, name, or date of birth. No agent’s reasoning depends on them.
- Trace payloads always mask SSN and name, and reduce date of birth to its year so age-driven decisions remain debuggable.
- Every string in a trace, including error messages, is scrubbed of SSN-shaped values as a backstop for fields added later.
- applicant_key is a plain SHA-256 of the SSN in this lab. Because SSNs have only about one billion values, production should use a keyed HMAC with the key held in a secrets manager.

Repeat submissions are handled in two ways:

- Same idempotency_key: the existing case is returned and no new processing starts. A unique index enforces one case per key.
- Same applicant with an open case (any status other than RECOMMENDATION_COMPLETED or PROCESSING_FAILED): a new case is processed, recorded in possible_duplicate_of, and routed to

  HUMAN_REVIEW so an underwriter sees both applications together.
- Same applicant with earlier approved cases: the new case is processed
  normally, but their approved coverage is added to the requested amount
  before the coverage-to-income rules apply.

Agent Access Boundaries

Agents have explicit responsibilities:

- Intake owns creation of the validated application.
- Enrichment owns external enrichment evidence.
- Risk Scoring owns the risk assessment.
- Recommendation owns the final recommendation.

Agents must not overwrite data owned by another stage.

LLM Memory

LLM calls are stateless.

The system never assumes that a model remembers previous calls. Required
context is explicitly constructed from the current case and supplied to
each model invocation.

Persistent business state belongs in the case repository, not in model
conversation history.

Production Evolution

The lab uses SQLite for lightweight durable persistence.

The repository interface separates domain logic from storage technology.
A production deployment could replace SQLite with a managed transactional
database and introduce asynchronous workers or event-driven processing
without redesigning the core case model.

Key Design Goals

The state architecture prioritizes:

1. Case isolation
2. Auditability
3. Explicit ownership
4. Persistence
5. Testability
6. Replaceable infrastructure
7. Prevention of cross-case context leakage

==========
2. Tracing implementation: trace record schema
Source: src/underwriting/observability/models.py
==========

```
from datetime import UTC, datetime
from enum import StrEnum
from typing import Any
from uuid import UUID

from pydantic import BaseModel, Field


class TraceStatus(StrEnum):
    """Execution outcome for one workflow stage attempt."""

    SUCCESS = "success"
    FAILED = "failed"


class TraceRecord(BaseModel):
    """
    Structured telemetry for one workflow-stage attempt.

    One record answers: which agent ran, when, what it read from shared
    state and where that input came from, what it wrote back, how long
    it took, what it cost, and whether it failed or was retried.

    received and produced are scrubbed of direct identifiers before
    they reach a trace store (see observability/payloads.py).

    estimated_cost_usd is the equivalent commercial-API cost of the
    call, not what local inference actually cost; see
    observability/cost.py.
    """

    case_id: UUID
    trace_id: UUID

    stage: str = Field(min_length=1)
    component: str | None = None
    attempt: int = Field(ge=1)
    status: TraceStatus

    started_at: datetime = Field(
        default_factory=lambda: datetime.now(UTC)
    )
    finished_at: datetime | None = None
    duration_ms: float = Field(ge=0)

    llm_latency_ms: float = Field(
        default=0.0,
        ge=0,
    )

    model: str | None = None
    input_tokens: int = Field(default=0, ge=0)
```

```
    output_tokens: int = Field(default=0, ge=0)

    estimated_cost_usd: float = Field(
        default=0.0,
        ge=0,
    )

    input_sources: dict[str, str] = Field(
        default_factory=dict
    )
    tools_used: list[str] = Field(
        default_factory=list
    )
    received: dict[str, Any] = Field(
        default_factory=dict
    )
    produced: dict[str, Any] = Field(
        default_factory=dict
    )
    state_writes: list[str] = Field(
        default_factory=list
    )

    error_type: str | None = None
    error_message: str | None = None

================================================================================
3. Tracing implementation: PII redaction
Source: src/underwriting/observability/payloads.py
================================================================================
import re
from datetime import date
from typing import Any

from pydantic import BaseModel

from underwriting.agents.enrichment import EnrichmentResult
from underwriting.agents.intake import IntakeResult
from underwriting.agents.recommendation import RecommendationResult
from underwriting.agents.risk_scoring import RiskScoringResult

PII_FIELDS = frozenset(
    {
        "ssn",
        "first_name",
        "last_name",
        "date_of_birth",
    }
)

REDACTED = "[REDACTED]"
REDACTED_SSN = "[REDACTED-SSN]"
```

```
_SSN_PATTERN = re.compile(r"\b\d{3}[- ]?\d{2}[- ]?\d{4}\b")
_ISO_DATE_PATTERN = re.compile(r"^(\d{4})-\d{2}-\d{2}")


def scrub_text(text: str) -> str:
    """Mask anything shaped like an SSN, wherever it appears in free text."""

    return _SSN_PATTERN.sub(REDACTED_SSN, text)


def _mask_field(key: str, value: Any) -> str:
    """
    Mask one identifier field.

    Date of birth keeps only the year so age-related decisions stay
    debuggable from the trace alone.
    """

    if key == "date_of_birth":
        if isinstance(value, date):
            return f"{value.year}-XX-XX"

        if isinstance(value, str):
            match = _ISO_DATE_PATTERN.match(value)

            if match:
                return f"{match.group(1)}-XX-XX"

    return REDACTED


def redact_pii(value: Any) -> Any:
    """
    Return a copy of a JSON-like payload that is safe to write to traces.

    Known identifier fields are masked at any depth, and SSN-shaped text
    is scrubbed from every string as a backstop for unexpected fields.
    """

    if isinstance(value, dict):
        return {
            key: (
                _mask_field(key, item)
                if key in PII_FIELDS
                else redact_pii(item)
            )
            for key, item in value.items()
        }

    if isinstance(value, list):
        return [redact_pii(item) for item in value]
```

```
    if isinstance(value, str):
        return scrub_text(value)

    return value


def _dump(value: Any) -> Any:
    if isinstance(value, BaseModel):
        return value.model_dump(mode="json")

    if isinstance(value, list):
        return [_dump(item) for item in value]

    if isinstance(value, dict):
        return {
            key: _dump(item)
            for key, item in value.items()
        }

    return value


def produced_payload(result: Any) -> dict[str, Any]:
    """
    Summarize what an agent produced for a workflow trace.

    LLM request/response bodies stay on llm_response; traces keep
    the business outputs the next stage consumes.
    """

    if isinstance(result, IntakeResult):
        return {
            "status": result.status.value,
            "application": _dump(result.application),
            "review": _dump(result.review),
            "validation_errors": list(
                result.validation_errors
            ),
        }

    if isinstance(result, EnrichmentResult):
        return {
            "status": result.status.value,
            "evidence": _dump(result.evidence),
            "discrepancies": list(result.discrepancies),
            "missing_evidence": list(
                result.missing_evidence
            ),
            "review": _dump(result.review),
        }
```

```
    if isinstance(result, RiskScoringResult):
        return {
            "assessment": _dump(result.assessment),
            "review": _dump(result.review),
        }

    if isinstance(result, RecommendationResult):
        return {
            "recommendation": _dump(
                result.recommendation
            ),
            "review": _dump(result.review),
        }

    raise TypeError(
        f"Unsupported agent result: {type(result)!r}"
    )
```

```
========================================================================
4. Full pipeline / state-sharing mechanism and trace wiring
Source: src/underwriting/workflow/orchestrator.py
========================================================================
```

```
from collections.abc import Callable
from dataclasses import dataclass
from datetime import UTC, date, datetime
from functools import partial
from time import perf_counter, sleep
from typing import Any, TypeVar

from underwriting.agents.enrichment import enrichment_agent
from underwriting.agents.intake import (
    IntakeStatus,
    intake_agent,
)
from underwriting.agents.recommendation import (
    recommendation_agent,
)
from underwriting.agents.risk_scoring import (
    risk_scoring_agent,
)
from underwriting.domain.application import Application
from underwriting.domain.case import (
    CaseMetadata,
    CaseState,
    CaseStatus,
)
from underwriting.enrichment.providers import (
    FinancialProvider,
    IdentityProvider,
    MedicalHistoryProvider,
    PrescriptionProvider,
)
```

```
from underwriting.llm.client import (
    LLMClient,
    LLMResponse,
)
from underwriting.observability.cost import (
    estimate_llm_cost_usd,
)
from underwriting.observability.models import (
    TraceRecord,
    TraceStatus,
)
from underwriting.observability.payloads import (
    produced_payload,
    redact_pii,
    scrub_text,
)
from underwriting.observability.trace_store import (
    TraceStore,
)
from underwriting.repositories.case_repository import (
    CaseAlreadyExistsError,
    CaseRepository,
)
from underwriting.workflow.transitions import (
    transition_case,
)

T = TypeVar("T")


class WorkflowExecutionError(Exception):
    """Raised when an underwriting workflow cannot continue."""


@dataclass(frozen=True)
class StageContract:
    """
    Data lineage for one workflow stage.

    input_sources maps each key of the stage's traced input to the
    component that produced it ("user_input" for the submission).
    state_writes lists the case-state fields a successful run fills.
    """

    name: str
    component: str
    input_sources: dict[str, str]
    state_writes: tuple[str, ...]


INTAKE_STAGE = StageContract(
    name="intake",
```

```
    component="intake_agent",
    input_sources={"raw_application": "user_input"},
    state_writes=("application",),
)

ENRICHMENT_STAGE = StageContract(
    name="enrichment",
    component="enrichment_agent",
    input_sources={"application": "intake_agent"},
    state_writes=("enrichment",),
)

RISK_SCORING_STAGE = StageContract(
    name="risk_scoring",
    component="risk_scoring_agent",
    input_sources={
        "application": "intake_agent",
        "evidence": "enrichment_agent",
        "existing_coverage": "case_repository",
    },
    state_writes=("risk_assessment",),
)

RECOMMENDATION_STAGE = StageContract(
    name="recommendation",
    component="recommendation_agent",
    input_sources={"assessment": "risk_scoring_agent"},
    state_writes=("recommendation",),
)


class _MeteredLLMClient(LLMClient):
    """
    Record every LLM call made during the current stage attempt.

    Usage is captured even when the stage later fails (for example on
    unparseable model output), so failed attempts are still costed.
    """

    def __init__(self, inner: LLMClient) -> None:
        self.inner = inner
        self.calls: list[LLMResponse] = []

    def generate(
        self,
        *,
        system_prompt: str,
        user_prompt: str,
    ) -> LLMResponse:
        response = self.inner.generate(
            system_prompt=system_prompt,
            user_prompt=user_prompt,
```

```
        )
        self.calls.append(response)
        return response


@dataclass(frozen=True)
class RetryPolicy:
    """
    Retry configuration for transient technical failures.

    max_attempts includes the initial attempt.
    """

    max_attempts: int = 3
    backoff_seconds: float = 0.0

    def __post_init__(self) -> None:
        if self.max_attempts < 1:
            raise ValueError(
                "max_attempts must be at least 1."
            )

        if self.backoff_seconds < 0:
            raise ValueError(
                "backoff_seconds cannot be negative."
            )


class UnderwritingOrchestrator:
    """
    Coordinate the underwriting workflow.

    Responsibilities:
    - sequence standalone agents
    - persist case state after successful stages
    - route business-review cases
    - retry explicitly retryable technical failures
    - persist terminal technical failures
    - emit structured per-attempt execution traces

    Underwriting policy remains inside the individual deterministic
    policy components, not inside this orchestrator.
    """

    def __init__(
        self,
        *,
        repository: CaseRepository,
        llm_client: LLMClient,
        identity_provider: IdentityProvider,
        medical_provider: MedicalHistoryProvider,
        prescription_provider: PrescriptionProvider,
```

```
    financial_provider: FinancialProvider,
    trace_store: TraceStore,
    retry_policy: RetryPolicy | None = None,
) -> None:
    self.repository = repository
    self.llm_client = llm_client
    self._metered_llm = _MeteredLLMClient(llm_client)
    self.identity_provider = identity_provider
    self.medical_provider = medical_provider
    self.prescription_provider = prescription_provider
    self.financial_provider = financial_provider
    self.trace_store = trace_store
    self.retry_policy = retry_policy or RetryPolicy()

def start_case(
    self,
    raw_application: dict[str, Any],
    *,
    evaluation_date: date | None = None,
    idempotency_key: str | None = None,
) -> CaseState:
    """
    Create and execute a new underwriting case.

    The case is persisted immediately so failures during the first
    processing stage still have a durable case record.

    A repeated submission with the same idempotency_key returns the
    case created by the first submission instead of starting another.
    """

    if idempotency_key is not None:
        existing = self.repository.find_by_idempotency_key(
            idempotency_key
        )

        if existing is not None:
            return existing

    case = CaseState(
        metadata=CaseMetadata(
            idempotency_key=idempotency_key,
        )
    )

    try:
        self.repository.create(case)

    except CaseAlreadyExistsError:
        existing = (
            self.repository.find_by_idempotency_key(
                idempotency_key
```

```
            )
            if idempotency_key is not None
            else None
        )

        if existing is None:
            raise

        return existing

    return self._execute(
        case=case,
        raw_application=raw_application,
        evaluation_date=evaluation_date or date.today(),
    )

def _execute(
    self,
    *,
    case: CaseState,
    raw_application: dict[str, Any],
    evaluation_date: date,
) -> CaseState:
    """
    Execute the underwriting workflow from intake through
    recommendation.
    """

    try:
        intake_result = self._with_retry(
            case=case,
            stage=INTAKE_STAGE,
            received={
                "raw_application": raw_application,
            },
            operation=lambda: intake_agent(
                raw_application=raw_application,
                llm_client=self._metered_llm,
            ),
            wrote_state=lambda result: (
                result.application is not None
            ),
        )

        if intake_result.status == IntakeStatus.INVALID:
            validation_message = "; ".join(
                intake_result.validation_errors
            )

            return self._fail_case(
                case,
                "Intake validation failed: "
```

```
        f"{validation_message}",
    )

if (
    intake_result.status
    == IntakeStatus.NEEDS_REVIEW
):
    if intake_result.application is not None:
        self._attach_application(
            case,
            intake_result.application,
        )

    concerns = (
        intake_result.review.concerns
        if intake_result.review is not None
        else []
    )

    case.errors.extend(
        f"Intake review: {concern}"
        for concern in concerns
    )

    case = transition_case(
        case,
        CaseStatus.HUMAN_REVIEW,
    )

    case = self.repository.save(case)

    if case.application is not None and (
        self._record_possible_duplicates(case)
    ):
        case = self.repository.save(case)

    return case

if intake_result.application is None:
    raise WorkflowExecutionError(
        "Accepted intake result did not contain "
        "a validated application."
    )

self._attach_application(
    case,
    intake_result.application,
)

case = transition_case(
    case,
    CaseStatus.INTAKE_COMPLETED,
```

```
)

case = self.repository.save(case)

self._record_possible_duplicates(case)

enrichment_result = self._with_retry(
    case=case,
    stage=ENRICHMENT_STAGE,
    received={
        "application": (
            case.application.model_dump(
                mode="json"
            )
        ),
    },
    operation=lambda: enrichment_agent(
        application=case.application,
        identity_provider=(
            self.identity_provider
        ),
        medical_provider=(
            self.medical_provider
        ),
        prescription_provider=(
            self.prescription_provider
        ),
        financial_provider=(
            self.financial_provider
        ),
        llm_client=self._metered_llm,
    ),
    tools_used=[
        type(provider).__name__
        for provider in (
            self.identity_provider,
            self.medical_provider,
            self.prescription_provider,
            self.financial_provider,
        )
    ],
)

case.enrichment = (
    enrichment_result.model_dump(
        mode="json"
    )
)

case = transition_case(
    case,
    CaseStatus.ENRICHMENT_COMPLETED,
```

```
)

case = self.repository.save(case)

existing_coverage = self._existing_coverage(case)

risk_result = self._with_retry(
    case=case,
    stage=RISK_SCORING_STAGE,
    received={
        "application": (
            case.application.model_dump(
                mode="json"
            )
        ),
        "evidence": (
            enrichment_result.evidence.model_dump(
                mode="json"
            )
        ),
        "existing_coverage": existing_coverage,
    },
    operation=lambda: risk_scoring_agent(
        application=case.application,
        evidence=enrichment_result.evidence,
        llm_client=self._metered_llm,
        evaluation_date=evaluation_date,
        existing_coverage=existing_coverage["amount"],
    ),
)

case.risk_assessment = (
    risk_result.assessment.model_dump(
        mode="json"
    )
)

case = transition_case(
    case,
    CaseStatus.RISK_ASSESSED,
)

case = self.repository.save(case)

recommendation_result = self._with_retry(
    case=case,
    stage=RECOMMENDATION_STAGE,
    received={
        "assessment": (
            risk_result.assessment.model_dump(
                mode="json"
            )
```

```
                ),
            },
            operation=lambda: recommendation_agent(
                assessment=risk_result.assessment,
                llm_client=self._metered_llm,
            ),
        )

        case.recommendation = (
            recommendation_result.model_dump(
                mode="json"
            )
        )

        if (
            recommendation_result
            .recommendation
            .requires_review
            or case.possible_duplicate_of
        ):
            case = transition_case(
                case,
                CaseStatus.HUMAN_REVIEW,
            )

        else:
            case = transition_case(
                case,
                CaseStatus.RECOMMENDATION_COMPLETED,
            )

        return self.repository.save(case)

    except (
        ConnectionError,
        TimeoutError,
        WorkflowExecutionError,
    ) as exc:
        if case.status in {
            CaseStatus.RECOMMENDATION_COMPLETED,
            CaseStatus.HUMAN_REVIEW,
            CaseStatus.PROCESSING_FAILED,
        }:
            raise

        return self._fail_case(
            case,
            "Workflow execution failed: "
            f"{type(exc).__name__}: {exc}",
        )

@staticmethod
```

```
def _attach_application(
    case: CaseState,
    application: Application,
) -> None:
    case.application = application
    case.metadata.applicant_key = application.applicant_key

def _record_possible_duplicates(
    self,
    case: CaseState,
) -> bool:
    """
    Flag other open cases for the same applicant.

    Runs after the case’s applicant key is persisted, so two
    concurrent submissions cannot both miss each other.
    """

    other_case_ids = [
        other.metadata.case_id
        for other in self.repository.find_open_by_applicant(
            case.metadata.applicant_key
        )
        if other.metadata.case_id != case.metadata.case_id
    ]

    if not other_case_ids:
        return False

    case.possible_duplicate_of = other_case_ids
    case.errors.append(
        "Possible duplicate of open case(s): "
        + ", ".join(str(case_id) for case_id in other_case_ids)
        + "."
    )

    return True

def _existing_coverage(
    self,
    case: CaseState,
) -> dict[str, Any]:
    """
    Total coverage from the applicant’s earlier approved cases.

    Pending cases are not counted here; they are already flagged as
    possible duplicates and routed to human review.
    """

    approved = self.repository.find_approved_by_applicant(
        case.metadata.applicant_key
    )
```

```
    return {
        "amount": sum(
            other.application.coverage_amount
            for other in approved
        ),
        "case_ids": [
            str(other.metadata.case_id)
            for other in approved
        ],
    }

def _with_retry(
    self,
    *,
    case: CaseState,
    stage: StageContract,
    received: dict[str, Any],
    operation: Callable[[], T],
    tools_used: list[str] | None = None,
    wrote_state: Callable[[T], bool] = lambda _: True,
) -> T:
    """
    Execute one workflow stage with bounded retries and tracing.

    Every attempt produces a TraceRecord.

    Only explicitly retryable technical exceptions are retried.
    Any other stage error (invalid LLM output, provider errors,
    programming defects) is traced once and converted into a
    WorkflowExecutionError so the case is persisted as failed.
    """

    last_error: Exception | None = None

    for attempt in range(
        1,
        self.retry_policy.max_attempts + 1,
    ):
        started_at = datetime.now(UTC)
        started_counter = perf_counter()
        self._metered_llm.calls.clear()

        record = partial(
            self._record_trace,
            case=case,
            stage=stage,
            attempt=attempt,
            started_at=started_at,
            started_counter=started_counter,
            tools_used=tools_used or [],
            received=received,
```

```
        )

        try:
            result = operation()

            record(
                status=TraceStatus.SUCCESS,
                produced=produced_payload(result),
                state_writes=(
                    stage.state_writes
                    if wrote_state(result)
                    else ()
                ),
            )

            return result

        except (
            ConnectionError,
            TimeoutError,
        ) as exc:
            last_error = exc

            record(status=TraceStatus.FAILED, error=exc)

            if (
                attempt
                < self.retry_policy.max_attempts
                and self.retry_policy.backoff_seconds > 0
            ):
                sleep(
                    self.retry_policy.backoff_seconds
                    * attempt
                )

        except Exception as exc:
            record(status=TraceStatus.FAILED, error=exc)

            raise WorkflowExecutionError(
                f"{stage.name} failed: "
                f"{type(exc).__name__}: {exc}"
            ) from exc

    raise WorkflowExecutionError(
        f"{stage.name} failed after "
        f"{self.retry_policy.max_attempts} attempts: "
        f"{last_error}"
    ) from last_error

def _record_trace(
    self,
    *,
```

```
    case: CaseState,
    stage: StageContract,
    attempt: int,
    started_at: datetime,
    started_counter: float,
    tools_used: list[str],
    received: dict[str, Any],
    status: TraceStatus,
    produced: dict[str, Any] | None = None,
    state_writes: tuple[str, ...] = (),
    error: Exception | None = None,
) -> None:
    """
    Record one stage attempt, with payloads scrubbed of PII.

    LLM usage covers every model call made during the attempt,
    including calls made before a failure.
    """

    duration_ms = (perf_counter() - started_counter) * 1000
    llm_calls = self._metered_llm.calls
    input_tokens = sum(call.input_tokens for call in llm_calls)
    output_tokens = sum(call.output_tokens for call in llm_calls)

    self.trace_store.record(
        TraceRecord(
            case_id=case.metadata.case_id,
            trace_id=case.metadata.trace_id,
            stage=stage.name,
            component=stage.component,
            attempt=attempt,
            status=status,
            started_at=started_at,
            finished_at=datetime.now(UTC),
            duration_ms=duration_ms,
            llm_latency_ms=sum(
                call.latency_ms for call in llm_calls
            ),
            model=(
                llm_calls[-1].model
                if llm_calls
                else None
            ),
            input_tokens=input_tokens,
            output_tokens=output_tokens,
            estimated_cost_usd=estimate_llm_cost_usd(
                input_tokens=input_tokens,
                output_tokens=output_tokens,
            ),
            input_sources=stage.input_sources,
            tools_used=tools_used,
            received=redact_pii(received),
```

```
                produced=redact_pii(produced or {}),
                state_writes=list(state_writes),
                error_type=(
                    type(error).__name__
                    if error is not None
                    else None
                ),
                error_message=(
                    scrub_text(str(error))
                    if error is not None
                    else None
                ),
            )
        )

    def _fail_case(
        self,
        case: CaseState,
        error_message: str,
    ) -> CaseState:
        """
        Persist a terminal technical workflow failure.
        """

        failed_case = case.model_copy(deep=True)

        failed_case.errors.append(
            error_message
        )

        failed_case = transition_case(
            failed_case,
            CaseStatus.PROCESSING_FAILED,
        )

        return self.repository.save(
            failed_case
        )
```

```
================================================================================
5. Cost and latency analysis
Source: docs/cost_latency_analysis.md
================================================================================
# Cost and Latency Analysis

This analysis uses traces from three mock applications
(‘case_01‘, ‘case_02‘, ‘case_03‘) run through the full
intake ? enrichment ? risk scoring ? recommendation
pipeline.

Local Ollama inference has a cash cost of **$0.00**.
Traces still record an **equivalent commercial cost** using
```

published gpt-4o-mini-class rates:

- $0.15 / 1M input tokens
- $0.60 / 1M output tokens

Those rates are implemented in
‘underwriting.observability.cost.estimate_llm_cost_usd‘.

The captured lab evidence uses a deterministic fake LLM so
the packet can be regenerated without a live model. Token
counts are therefore representative of a small structured
JSON call (420 input / 90 output per agent), not a long
chat session. Wall-clock duration in that mode is
sub-second. A live Ollama run on ‘qwen3:8b‘ is typically
8?40 seconds per case depending on host load; that is
the latency figure used for the real-time vs batch judgment.

Per-case totals from captured traces

| Case | Stages | Input tokens | Output tokens | Equivalent USD | Stage duration (fake LLM)
|
|---|---|---|---|---|---|
| case_01 | 4 | 1680 | 360 | $0.000468 | < 1 s |
| case_02 | 4 | 1680 | 360 | $0.000468 | < 1 s |
| case_03 | 4 | 1680 | 360 | $0.000468 | < 1 s (routes to human_review) |

Formula used for each stage attempt (failed attempts are
costed too, for every model call that returned before the
failure):

‘‘‘
cost = (input_tokens / 1e6) * 0.15 + (output_tokens / 1e6) * 0.60
‘‘‘

Four successful calls × (420, 90) tokens:

‘‘‘
4 * ((420 / 1e6) * 0.15 + (90 / 1e6) * 0.60) = 0.000468
‘‘‘

Real-time vs overnight batch

**Not acceptable as a hard real-time / interactive
underwriting API** if the product needs a sub-second or
even 2-second response. Four sequential LLM calls add
serial latency. On a local 8B model that commonly lands
in the **tens of seconds** per application. That is fine
for a human underwriter waiting on a workstation, but not
for a quote-in-the-browser experience.

Acceptable for overnight / batch underwriting. Cost is
negligible even at commercial rates (well under a tenth of

a cent per case). Overnight jobs routinely spend minutes
per file when humans or third-party data vendors are in
the loop. Retry-then-escalate also fits batch: a timed-out
risk score becomes ‘processing_failed‘ for a human queue
instead of a silent wrong decision.

Borderline for same-day assisted underwriting. If a
human reviewer already waits 5?15 minutes for MIB /
prescription / financial hits, adding ~30 seconds of model
time is reasonable. The system should still show progress
and never return an incomplete recommendation as if it
were final.

Failure-path cost note

When risk scoring times out three times, the case is
flagged ‘processing_failed‘. Intake and enrichment tokens
are still spent; risk scoring adds duration but $0 extra
because those attempts never returned a completion. That
is cheaper than inventing a score.

```
========================================================================
6. Test and coverage output
Source: artifacts/lab_evidence/unit_tests.txt
========================================================================
ssssssss.............................................................. [ 19%]
.............................................................................. [ 39%]
.............................................................................. [ 59%]
.............................................................................. [ 79%]
.............................................................................. [ 99%]
.                                                                             [100%]
================================ tests coverage ================================
_______________ coverage: platform linux, python 3.12.15-final-0 _______________

Name                                                                                  Stmts   Miss  Cover   Missing
-------------------------------------------------------------------------------------------------------------------
src/underwriting/__init__.py                                                              0      0   100%
src/underwriting/agents/__init__.py                                                       0      0   100%
src/underwriting/agents/enrichment.py                                                    92      0   100%
src/underwriting/agents/intake.py                                                        56      0   100%
src/underwriting/agents/recommendation.py                                                30      0   100%
src/underwriting/agents/risk_scoring.py                                                  33      0   100%
src/underwriting/domain/__init__.py                                                       0      0   100%
src/underwriting/domain/application.py                                                   73      0   100%
src/underwriting/domain/case.py                                                          33      0   100%
src/underwriting/enrichment/__init__.py                                                   0      0   100%
src/underwriting/enrichment/matching.py                                                  18      0   100%
src/underwriting/enrichment/models.py                                                    33      0   100%
src/underwriting/enrichment/providers.py                                                 18      0   100%
src/underwriting/enrichment/synthetic_providers.py                                       60      0   100%
src/underwriting/enrichment/synthetic_store.py                                           24      0   100%
src/underwriting/llm/__init__.py                                                          0      0   100%
```

```
src/underwriting/llm/client.py                                  11      0   100%
src/underwriting/llm/ollama_client.py                           34      0   100%
src/underwriting/observability/__init__.py                       0      0   100%
src/underwriting/observability/cost.py                           8      0   100%
src/underwriting/observability/models.py                        30      0   100%
src/underwriting/observability/payloads.py                      50      0   100%
src/underwriting/observability/sqlite_trace_store.py            44      0   100%
src/underwriting/observability/trace_store.py                   15      0   100%
src/underwriting/recommendation/models.py                       19      0   100%
src/underwriting/recommendation/policy.py                       13      0   100%
src/underwriting/repositories/__init__.py                        0      0   100%
src/underwriting/repositories/case_repository.py                19      0   100%
src/underwriting/repositories/sqlite_case_repository.py         76      0   100%
src/underwriting/risk/__init__.py                                0      0   100%
src/underwriting/risk/models.py                                 39      0   100%
src/underwriting/risk/policy.py                                142      0   100%
src/underwriting/workflow/__init__.py                            0      0   100%
src/underwriting/workflow/orchestrator.py                      160      0   100%
src/underwriting/workflow/transitions.py                        11      0   100%
-------------------------------------------------------------------------------------
TOTAL                                                         1141      0   100%
Required test coverage of 100.0% reached. Total coverage: 100.00%
353 passed, 8 skipped in 1.09s

======================================================================
7. Trace: case_01
Source: artifacts/lab_evidence/case_01_trace.txt
======================================================================
======================================================================
CASE: case_01
======================================================================
case_id: d1264fd5-9f9c-4903-b62e-d7498aa47c02
trace_id: fe20cce4-c0c8-4aed-9b42-0a30045c2635
status: recommendation_completed

TRACE
intake           component=intake_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.668546+00:00 duration_ms=0.14 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"raw_application": "user_input"}
  received={"raw_application": {"ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1990-XX-XX", "state": "PA", "annual_income":
90000, "coverage_amount": 500000, "occupation": "Software Engineer", "tobacco_use": "never",
 "height_cm": 175, "weight_kg": 75, "medical_conditions_declared": [],
"medications_declared": []}}
  produced={"status": "accepted", "application": {"application_id":
"ff853c07-dc79-4151-bc92-65d0179bb7fb", "ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1990-XX-XX", "state": "PA", "annual_income":
90000, "coverage_amount": 500000, "occupation": "Software Engineer", "tobacco_use": "never",
 "height_cm": 175, "weight_kg": 75.0, "medical_conditions_declared": [],
"medications_declared": []}, "review": {"needs_review": false, "summary": "Applicant-
provided information is clear.", "concerns": []}, "validation_errors": []}
```

```
  state_writes=["application"]
enrichment       component=enrichment_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.672888+00:00 duration_ms=0.87 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent"}
  tools_used=SyntheticIdentityProvider, SyntheticMedicalHistoryProvider,
SyntheticPrescriptionProvider, SyntheticFinancialProvider
  received={"application": {"application_id": "ff853c07-dc79-4151-bc92-65d0179bb7fb", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000,
"occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg":
75.0, "medical_conditions_declared": [], "medications_declared": []}}
  produced={"status": "completed", "evidence": {"identity": {"status": "available",
"identity_verified": true, "name_match": true, "date_of_birth_match": true, "state_match":
true}, "medical": {"status": "available", "conditions": []}, "prescription": {"status":
"available", "prescriptions": []}, "financial": {"status": "available", "income_verified":
true, "verified_annual_income": 90000}}, "discrepancies": [], "missing_evidence": [],
"review": {"evidence_consistency": "consistent", "summary": "External evidence is consistent
 with the application.", "material_concerns": [], "corroborating_evidence": ["Identity and
financial records match."]}}
  state_writes=["enrichment"]
risk_scoring     component=risk_scoring_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.677428+00:00 duration_ms=0.21 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "ff853c07-dc79-4151-bc92-65d0179bb7fb", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000,
"occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg":
75.0, "medical_conditions_declared": [], "medications_declared": []}, "evidence":
{"identity": {"status": "available", "identity_verified": true, "name_match": true,
"date_of_birth_match": true, "state_match": true}, "medical": {"status": "available",
"conditions": []}, "prescription": {"status": "available", "prescriptions": []},
"financial": {"status": "available", "income_verified": true, "verified_annual_income":
90000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 36, "bmi": 24.49, "existing_coverage": 0, "coverage_to_income_ratio":
5.56, "score": 5, "tier": "low", "risk_factors": [{"code": "COVERAGE_RATIO_5_TO_8",
"description": "Coverage-to-income ratio is greater than 5 and no greater than 8.",
"points": 5}], "review_flags": [], "requires_review": false}, "review": {"summary":
"Deterministic risk assessment synthesized.", "primary_drivers": ["age", "coverage"],
"factor_interactions": [], "evidence_context": [], "review_context": []}}
  state_writes=["risk_assessment"]
recommendation   component=recommendation_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.680785+00:00 duration_ms=0.06 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"assessment": "risk_scoring_agent"}
  received={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 36, "bmi": 24.49, "existing_coverage": 0, "coverage_to_income_ratio":
5.56, "score": 5, "tier": "low", "risk_factors": [{"code": "COVERAGE_RATIO_5_TO_8",
"description": "Coverage-to-income ratio is greater than 5 and no greater than 8.",
```

"points": 5}], "review_flags": [], "requires_review": false}}
  produced={"recommendation": {"recommendation_policy_version": "synthetic-
recommendation-v1", "risk_policy_version": "synthetic-life-v3", "decision": "approve",
"reason_code": "automated_approval", "risk_score": 5, "risk_tier": "low", "requires_review":
 false}, "review": {"summary": "Authoritative recommendation follows the risk assessment.",
"decision_basis": ["tier", "score"], "precedence_explanation": null, "human_review_focus":
[]}}
  state_writes=["recommendation"]

total_stage_duration_ms: 1.27
total_input_tokens: 1680
total_output_tokens: 360
total_equiv_cloud_cost_usd (local inference is free): 0.00046800

==========
8. Trace: case_02
Source: artifacts/lab_evidence/case_02_trace.txt
==========
==========
CASE: case_02
==========
case_id: 0ac5555f-ff03-47ef-8891-04a91db71540
trace_id: 5ee3ea6c-1e48-494c-8b43-b62e989b0b39
status: recommendation_completed

TRACE
intake   component=intake_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.689225+00:00 duration_ms=0.09 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"raw_application": "user_input"}
  received={"raw_application": {"ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1982-XX-XX", "state": "NJ", "annual_income":
120000, "coverage_amount": 750000, "occupation": "Project Manager", "tobacco_use": "never",
"height_cm": 165, "weight_kg": 72, "medical_conditions_declared": ["Hypertension"],
"medications_declared": ["Lisinopril"]}}
  produced={"status": "accepted", "application": {"application_id": "4968fe5a-7c32-46cb-
bafd-a81d20657ea8", "ssn": "[REDACTED]", "first_name": "[REDACTED]", "last_name":
"[REDACTED]", "date_of_birth": "1982-XX-XX", "state": "NJ", "annual_income": 120000,
"coverage_amount": 750000, "occupation": "Project Manager", "tobacco_use": "never",
"height_cm": 165, "weight_kg": 72.0, "medical_conditions_declared": ["Hypertension"],
"medications_declared": ["Lisinopril"]}, "review": {"needs_review": false, "summary":
"Applicant-provided information is clear.", "concerns": []}, "validation_errors": []}
  state_writes=["application"]
enrichment   component=enrichment_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.693117+00:00 duration_ms=0.85 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent"}
  tools_used=SyntheticIdentityProvider, SyntheticMedicalHistoryProvider,
SyntheticPrescriptionProvider, SyntheticFinancialProvider
  received={"application": {"application_id": "4968fe5a-7c32-46cb-bafd-a81d20657ea8", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1982-XX-XX", "state": "NJ", "annual_income": 120000, "coverage_amount": 750000,

"occupation": "Project Manager", "tobacco_use": "never", "height_cm": 165, "weight_kg":
72.0, "medical_conditions_declared": ["Hypertension"], "medications_declared":
["Lisinopril"]}}
  produced={"status": "completed", "evidence": {"identity": {"status": "available",
"identity_verified": true, "name_match": true, "date_of_birth_match": true, "state_match":
true}, "medical": {"status": "available", "conditions": [{"condition": "Hypertension",
"status": "managed"}]}, "prescription": {"status": "available", "prescriptions":
[{"medication": "Lisinopril", "active": true}]}, "financial": {"status": "available",
"income_verified": true, "verified_annual_income": 120000}}, "discrepancies": [],
"missing_evidence": [], "review": {"evidence_consistency": "consistent", "summary":
"External evidence is consistent with the application.", "material_concerns": [],
"corroborating_evidence": ["Identity and financial records match."]}}
  state_writes=["enrichment"]
risk_scoring   component=risk_scoring_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.697311+00:00 duration_ms=0.10 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "4968fe5a-7c32-46cb-bafd-a81d20657ea8", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1982-XX-XX", "state": "NJ", "annual_income": 120000, "coverage_amount": 750000,
"occupation": "Project Manager", "tobacco_use": "never", "height_cm": 165, "weight_kg":
72.0, "medical_conditions_declared": ["Hypertension"], "medications_declared":
["Lisinopril"]}, "evidence": {"identity": {"status": "available", "identity_verified": true,
 "name_match": true, "date_of_birth_match": true, "state_match": true}, "medical":
{"status": "available", "conditions": [{"condition": "Hypertension", "status": "managed"}]},
 "prescription": {"status": "available", "prescriptions": [{"medication": "Lisinopril",
"active": true}]}, "financial": {"status": "available", "income_verified": true,
"verified_annual_income": 120000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 44, "bmi": 26.45, "existing_coverage": 0, "coverage_to_income_ratio":
6.25, "score": 20, "tier": "moderate", "risk_factors": [{"code": "AGE_40_49", "description":
 "Applicant age is between 40 and 49.", "points": 5}, {"code": "COVERAGE_RATIO_5_TO_8",
"description": "Coverage-to-income ratio is greater than 5 and no greater than 8.",
"points": 5}, {"code": "MEDICAL_HYPERTENSION", "description": "External medical evidence
contains Hypertension.", "points": 10}], "review_flags": [], "requires_review": false},
"review": {"summary": "Deterministic risk assessment synthesized.", "primary_drivers":
["age", "coverage"], "factor_interactions": [], "evidence_context": [], "review_context":
[]}}
  state_writes=["risk_assessment"]
recommendation   component=recommendation_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.700537+00:00 duration_ms=0.05 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"assessment": "risk_scoring_agent"}
  received={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 44, "bmi": 26.45, "existing_coverage": 0, "coverage_to_income_ratio":
6.25, "score": 20, "tier": "moderate", "risk_factors": [{"code": "AGE_40_49", "description":
 "Applicant age is between 40 and 49.", "points": 5}, {"code": "COVERAGE_RATIO_5_TO_8",
"description": "Coverage-to-income ratio is greater than 5 and no greater than 8.",
"points": 5}, {"code": "MEDICAL_HYPERTENSION", "description": "External medical evidence
contains Hypertension.", "points": 10}], "review_flags": [], "requires_review": false}}
  produced={"recommendation": {"recommendation_policy_version": "synthetic-

recommendation-v1", "risk_policy_version": "synthetic-life-v3", "decision": "approve",
"reason_code": "automated_approval", "risk_score": 20, "risk_tier": "moderate",
"requires_review": false}, "review": {"summary": "Authoritative recommendation follows the
risk assessment.", "decision_basis": ["tier", "score"], "precedence_explanation": null,
"human_review_focus": []}}
  state_writes=["recommendation"]

total_stage_duration_ms: 1.08
total_input_tokens: 1680
total_output_tokens: 360
total_equiv_cloud_cost_usd (local inference is free): 0.00046800

==========
9. Trace: case_03
Source: artifacts/lab_evidence/case_03_trace.txt
==========
==========
CASE: case_03
==========
case_id: eba24895-7cbf-460c-a8bf-8a7a74d0176a
trace_id: 770ac0e9-5d08-4dc4-a992-e583b899bbae
status: human_review

TRACE
intake   component=intake_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.707551+00:00 duration_ms=0.06 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"raw_application": "user_input"}
  received={"raw_application": {"ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1978-XX-XX", "state": "NY", "annual_income":
95000, "coverage_amount": 1000000, "occupation": "Sales Director", "tobacco_use": "never",
"height_cm": 180, "weight_kg": 96, "medical_conditions_declared": [],
"medications_declared": []}}
  produced={"status": "accepted", "application": {"application_id":
"758fa8a1-3538-434c-8872-9a053345ad4a", "ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1978-XX-XX", "state": "NY", "annual_income":
95000, "coverage_amount": 1000000, "occupation": "Sales Director", "tobacco_use": "never",
"height_cm": 180, "weight_kg": 96.0, "medical_conditions_declared": [],
"medications_declared": []}, "review": {"needs_review": false, "summary": "Applicant-
provided information is clear.", "concerns": []}, "validation_errors": []}
  state_writes=["application"]
enrichment   component=enrichment_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.711599+00:00 duration_ms=0.84 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent"}
  tools_used=SyntheticIdentityProvider, SyntheticMedicalHistoryProvider,
SyntheticPrescriptionProvider, SyntheticFinancialProvider
  received={"application": {"application_id": "758fa8a1-3538-434c-8872-9a053345ad4a", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1978-XX-XX", "state": "NY", "annual_income": 95000, "coverage_amount": 1000000,
"occupation": "Sales Director", "tobacco_use": "never", "height_cm": 180, "weight_kg": 96.0,
 "medical_conditions_declared": [], "medications_declared": []}}

```
  produced={"status": "needs_review", "evidence": {"identity": {"status": "conflict",
"identity_verified": false, "name_match": true, "date_of_birth_match": true, "state_match":
false}, "medical": {"status": "available", "conditions": [{"condition": "Type 2 Diabetes",
"status": "managed"}]}, "prescription": {"status": "available", "prescriptions":
[{"medication": "Metformin", "active": true}]}, "financial": {"status": "conflict",
"income_verified": false, "verified_annual_income": 55000}}, "discrepancies": ["Identity
evidence conflicts with the application.", "External medical evidence contains an undeclared
 condition: Type 2 Diabetes.", "External prescription evidence contains an undeclared active
 medication: Metformin.", "Financial evidence conflicts with the application."],
"missing_evidence": [], "review": {"evidence_consistency": "consistent", "summary":
"External evidence is consistent with the application.", "material_concerns": [],
"corroborating_evidence": ["Identity and financial records match."]}}
  state_writes=["enrichment"]
risk_scoring     component=risk_scoring_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.715716+00:00 duration_ms=0.14 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "758fa8a1-3538-434c-8872-9a053345ad4a", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1978-XX-XX", "state": "NY", "annual_income": 95000, "coverage_amount": 1000000,
"occupation": "Sales Director", "tobacco_use": "never", "height_cm": 180, "weight_kg": 96.0,
 "medical_conditions_declared": [], "medications_declared": []}, "evidence": {"identity":
{"status": "conflict", "identity_verified": false, "name_match": true,
"date_of_birth_match": true, "state_match": false}, "medical": {"status": "available",
"conditions": [{"condition": "Type 2 Diabetes", "status": "managed"}]}, "prescription":
{"status": "available", "prescriptions": [{"medication": "Metformin", "active": true}]},
"financial": {"status": "conflict", "income_verified": false, "verified_annual_income":
55000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 48, "bmi": 29.63, "existing_coverage": 0, "coverage_to_income_ratio":
10.53, "score": 25, "tier": "moderate", "risk_factors": [{"code": "AGE_40_49",
"description": "Applicant age is between 40 and 49.", "points": 5}, {"code":
"MEDICAL_TYPE_2_DIABETES", "description": "External medical evidence contains Type 2
Diabetes.", "points": 20}], "review_flags": ["identity_conflict", "financial_conflict",
"undeclared_medical_condition", "undeclared_active_medication", "excessive_coverage_ratio"],
 "requires_review": true}, "review": {"summary": "Deterministic risk assessment
synthesized.", "primary_drivers": ["age", "coverage"], "factor_interactions": [],
"evidence_context": [], "review_context": []}}
  state_writes=["risk_assessment"]
recommendation   component=recommendation_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.718694+00:00 duration_ms=0.05 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"assessment": "risk_scoring_agent"}
  received={"assessment": {"policy_version": "synthetic-life-v3", "evaluation_date":
"2026-10-08", "age": 48, "bmi": 29.63, "existing_coverage": 0, "coverage_to_income_ratio":
10.53, "score": 25, "tier": "moderate", "risk_factors": [{"code": "AGE_40_49",
"description": "Applicant age is between 40 and 49.", "points": 5}, {"code":
"MEDICAL_TYPE_2_DIABETES", "description": "External medical evidence contains Type 2
Diabetes.", "points": 20}], "review_flags": ["identity_conflict", "financial_conflict",
"undeclared_medical_condition", "undeclared_active_medication", "excessive_coverage_ratio"],
 "requires_review": true}}
```

  produced={"recommendation": {"recommendation_policy_version": "synthetic-
recommendation-v1", "risk_policy_version": "synthetic-life-v3", "decision": "refer",
"reason_code": "mandatory_review", "risk_score": 25, "risk_tier": "moderate",
"requires_review": true}, "review": {"summary": "Authoritative recommendation follows the
risk assessment.", "decision_basis": ["tier", "score"], "precedence_explanation": null,
"human_review_focus": []}}
  state_writes=["recommendation"]

total_stage_duration_ms: 1.09
total_input_tokens: 1680
total_output_tokens: 360
total_equiv_cloud_cost_usd (local inference is free): 0.00046800

==========
10. Simulated failure (retry then escalate)
Source: artifacts/lab_evidence/failure_trace.txt
==========
==========
CASE: case_01_timeout_risk_scoring
==========
case_id: 5e345392-7e4c-4bba-b9c2-b96876e0f163
trace_id: 1f8f8ef8-af97-4425-a435-8f9af0b1b786
status: processing_failed

TRACE
intake   component=intake_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.731481+00:00 duration_ms=0.08 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"raw_application": "user_input"}
  received={"raw_application": {"ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1990-XX-XX", "state": "PA", "annual_income":
90000, "coverage_amount": 500000, "occupation": "Software Engineer", "tobacco_use": "never",
 "height_cm": 175, "weight_kg": 75, "medical_conditions_declared": [],
"medications_declared": []}}
  produced={"status": "accepted", "application": {"application_id":
"dcbc3d31-8428-4c5f-bdec-2172b3808672", "ssn": "[REDACTED]", "first_name": "[REDACTED]",
"last_name": "[REDACTED]", "date_of_birth": "1990-XX-XX", "state": "PA", "annual_income":
90000, "coverage_amount": 500000, "occupation": "Software Engineer", "tobacco_use": "never",
 "height_cm": 175, "weight_kg": 75.0, "medical_conditions_declared": [],
"medications_declared": []}, "review": {"needs_review": false, "summary": "Applicant-
provided information is clear.", "concerns": []}, "validation_errors": []}
  state_writes=["application"]
enrichment   component=enrichment_agent attempt=1 status=success
started_at=2026-10-13T07:34:04.734806+00:00 duration_ms=0.95 tokens=420/90
equiv_cloud_cost_usd=0.00011700
  input_sources={"application": "intake_agent"}
  tools_used=SyntheticIdentityProvider, SyntheticMedicalHistoryProvider,
SyntheticPrescriptionProvider, SyntheticFinancialProvider
  received={"application": {"application_id": "dcbc3d31-8428-4c5f-bdec-2172b3808672", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000,
"occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg":

75.0, "medical_conditions_declared": [], "medications_declared": []}}
  produced={"status": "completed", "evidence": {"identity": {"status": "available",
"identity_verified": true, "name_match": true, "date_of_birth_match": true, "state_match":
true}, "medical": {"status": "available", "conditions": []}, "prescription": {"status":
"available", "prescriptions": []}, "financial": {"status": "available", "income_verified":
true, "verified_annual_income": 90000}}, "discrepancies": [], "missing_evidence": [],
"review": {"evidence_consistency": "consistent", "summary": "External evidence is consistent
 with the application.", "material_concerns": [], "corroborating_evidence": ["Identity and
financial records match."]}}
  state_writes=["enrichment"]
risk_scoring   component=risk_scoring_agent attempt=1 status=failed
started_at=2026-10-13T07:34:04.739402+00:00 duration_ms=0.07 tokens=0/0
equiv_cloud_cost_usd=0.00000000
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "dcbc3d31-8428-4c5f-bdec-2172b3808672", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000,
"occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg":
75.0, "medical_conditions_declared": [], "medications_declared": []}, "evidence":
{"identity": {"status": "available", "identity_verified": true, "name_match": true,
"date_of_birth_match": true, "state_match": true}, "medical": {"status": "available",
"conditions": []}, "prescription": {"status": "available", "prescriptions": []},
"financial": {"status": "available", "income_verified": true, "verified_annual_income":
90000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={}
  state_writes=[]
  error=TimeoutError: Synthetic risk-scoring timeout.
risk_scoring   component=risk_scoring_agent attempt=2 status=failed
started_at=2026-10-13T07:34:04.741048+00:00 duration_ms=0.07 tokens=0/0
equiv_cloud_cost_usd=0.00000000
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "dcbc3d31-8428-4c5f-bdec-2172b3808672", "ssn":
 "[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth":
"1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000,
"occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg":
75.0, "medical_conditions_declared": [], "medications_declared": []}, "evidence":
{"identity": {"status": "available", "identity_verified": true, "name_match": true,
"date_of_birth_match": true, "state_match": true}, "medical": {"status": "available",
"conditions": []}, "prescription": {"status": "available", "prescriptions": []},
"financial": {"status": "available", "income_verified": true, "verified_annual_income":
90000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={}
  state_writes=[]
  error=TimeoutError: Synthetic risk-scoring timeout.
risk_scoring   component=risk_scoring_agent attempt=3 status=failed
started_at=2026-10-13T07:34:04.742831+00:00 duration_ms=0.06 tokens=0/0
equiv_cloud_cost_usd=0.00000000
  input_sources={"application": "intake_agent", "evidence": "enrichment_agent",
"existing_coverage": "case_repository"}
  received={"application": {"application_id": "dcbc3d31-8428-4c5f-bdec-2172b3808672", "ssn":

"[REDACTED]", "first_name": "[REDACTED]", "last_name": "[REDACTED]", "date_of_birth": "1990-XX-XX", "state": "PA", "annual_income": 90000, "coverage_amount": 500000, "occupation": "Software Engineer", "tobacco_use": "never", "height_cm": 175, "weight_kg": 75.0, "medical_conditions_declared": [], "medications_declared": []}, "evidence": {"identity": {"status": "available", "identity_verified": true, "name_match": true, "date_of_birth_match": true, "state_match": true}, "medical": {"status": "available", "conditions": []}, "prescription": {"status": "available", "prescriptions": []}, "financial": {"status": "available", "income_verified": true, "verified_annual_income": 90000}}, "existing_coverage": {"amount": 0, "case_ids": []}}
  produced={}
  state_writes=[]
  error=TimeoutError: Synthetic risk-scoring timeout.

total_stage_duration_ms: 1.23
total_input_tokens: 840
total_output_tokens: 180
total_equiv_cloud_cost_usd (local inference is free): 0.00023400
errors:
  - Workflow execution failed: WorkflowExecutionError: risk_scoring failed after 3 attempts: Synthetic risk-scoring timeout.

==
11. CI workflow
Source: .github/workflows/ci.yml
==
```
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  quality:
    name: Lint and Tests
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"

      - name: Install dependencies
        run: |
```

```
    python -m pip install --upgrade pip
    pip install -e ".[dev]"

- name: Run Ruff
  run: |
    ruff check src tests

- name: Run tests with coverage
  run: |
    pytest -q --cov
```